

Serverless Web Application with AWS Lambda and API Gateway

Project: Serverless Web Application with AWS Lambda and API Gateway

Services: AWS Lambda, Amazon API Gateway, Amazon S3, Amazon DynamoDB, Amazon Cognito

Description: Developed a fully serverless web application that allows authenticated users to upload files securely to Amazon S3. User authentication is managed using Amazon Cognito, API requests are handled through Amazon API Gateway, backend processing is performed using AWS Lambda, and file metadata is stored in Amazon DynamoDB.

Skills: Serverless Architecture, API Integration, Authentication, Cloud Storage, NoSQL Database Management, AWS Lambda Scripting, Frontend Integration

Definition: The Serverless File Upload Web Application is designed to provide a secure and scalable platform for authenticated users to upload files through a web interface. The application uses Amazon Cognito for authentication, Amazon API Gateway for handling REST API requests, AWS Lambda for backend processing, Amazon S3 for file storage, and Amazon DynamoDB for storing file metadata.

The application follows a fully serverless architecture, eliminating the need for managing servers while ensuring scalability, security, and cost optimization.

Scope of the Project: The scope of this project is to design and implement a fully serverless file upload application using AWS managed services. The application enables secure file uploads, stores uploaded files in Amazon S3, and maintains file metadata in DynamoDB.

This project covers:

- User Authentication using Amazon Cognito.
- File Upload using API Gateway and Lambda.
- Secure File Storage in Amazon S3.
- Metadata Storage in DynamoDB.
- REST API Integration using API Gateway.
- Fully Serverless Backend Architecture.
- Browser-based Frontend Interface using HTML, CSS, and JavaScript.

Methodology:

1. Lambda + API Gateway + S3 + DynamoDB + Cognito – Best Method

- Uses all required AWS services: Lambda, API Gateway, S3, DynamoDB, and Cognito.
- Provides secure authentication and authorization using Cognito Hosted UI.
- Fully serverless architecture with no server management required.
- Scalable and production-ready architecture suitable for organizational deployment.
- Files are securely uploaded and stored in Amazon S3.
- Metadata such as filename, upload time, and file URL are stored in DynamoDB.
- REST APIs are managed using API Gateway.

- Cost-effective and AWS Free Tier friendly for small workloads.
- Provides high availability, scalability, and security using AWS managed services.

2. Traditional Server-Based File Upload – Alternative Method

- Uses EC2 instances and traditional backend servers for file uploads.
- Requires manual server setup, maintenance, patching, and scaling.
- File uploads and metadata management are handled using application servers.
- Involves higher infrastructure and operational costs compared to serverless architecture.
- Requires continuous server monitoring and maintenance.
- Suitable for legacy systems or applications requiring persistent backend servers.

AWS Services used in the Project:

1. Amazon S3 (Simple Storage Service)

Definition: Amazon S3 is a scalable object storage service used to store and retrieve any amount of data from anywhere on the web.

Purpose: Stores uploaded files securely in cloud storage.

Role:

- Stores uploaded user files.
- Maintains uploaded file objects in the uploads/ folder.
- Provides scalable and durable file storage.

2. Amazon DynamoDB

Definition: Amazon DynamoDB is a fully managed NoSQL database service that provides fast and predictable performance with seamless scalability.

Purpose: Stores metadata information about uploaded files.

Role: Stores file metadata such as:

- fileId
- fileName
- uploadTime
- fileUrl
- Enables fast retrieval of uploaded file details.

3. AWS Lambda

Definition: AWS Lambda is a serverless compute service that allows users to run backend code without provisioning or managing servers.

Purpose: Processes file uploads and backend operations.

Role:

- Receives API requests from API Gateway.
- Decodes uploaded file content.
- Uploads files to Amazon S3.
- Stores metadata in DynamoDB.
- Returns API responses to frontend applications.

4. Amazon API Gateway

Definition: Amazon API Gateway is a managed service that allows developers to create, publish, maintain, monitor, and secure REST APIs.

Purpose: Acts as the communication layer between frontend and Lambda.

Role:

- Receives HTTP POST requests from frontend applications.
- Invokes Lambda functions.
- Exposes REST API endpoints.
- Handles CORS configuration for browser requests.

5. Amazon Cognito

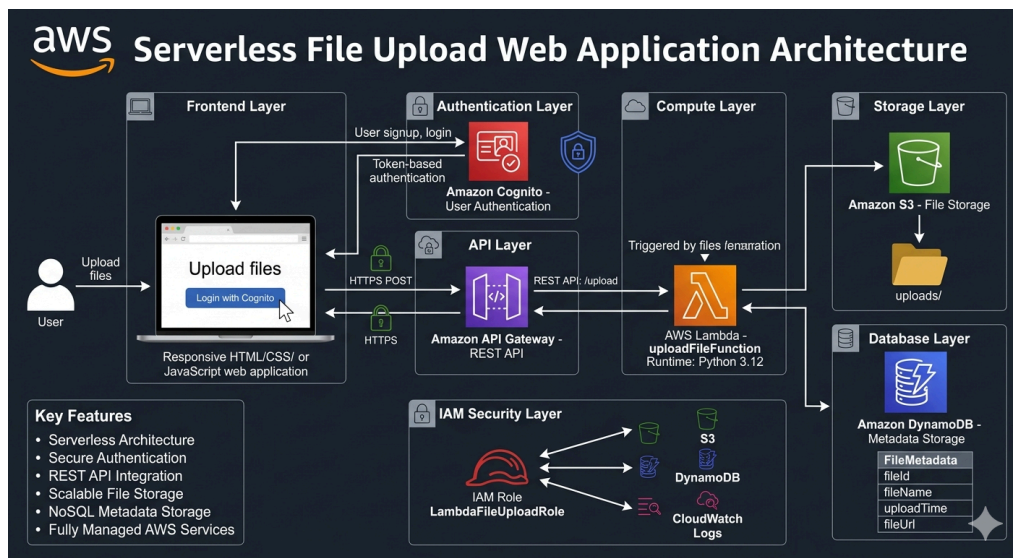
Definition: Amazon Cognito is a user authentication and authorization service that enables secure user signup and login functionality.

Purpose: Handles user authentication and authorization.

Role:

- Provides user signup and login functionality.
- Uses Cognito Hosted UI for authentication.
- Generates authentication tokens after successful login.
- Secures application access for authenticated users.

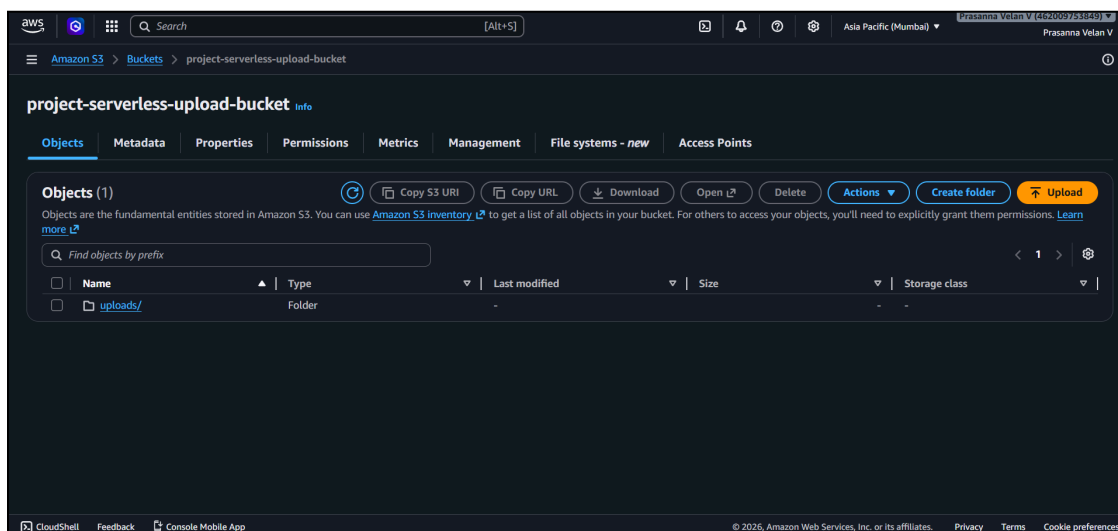
Architecture Diagram:



Implementation Steps to Do the Project:

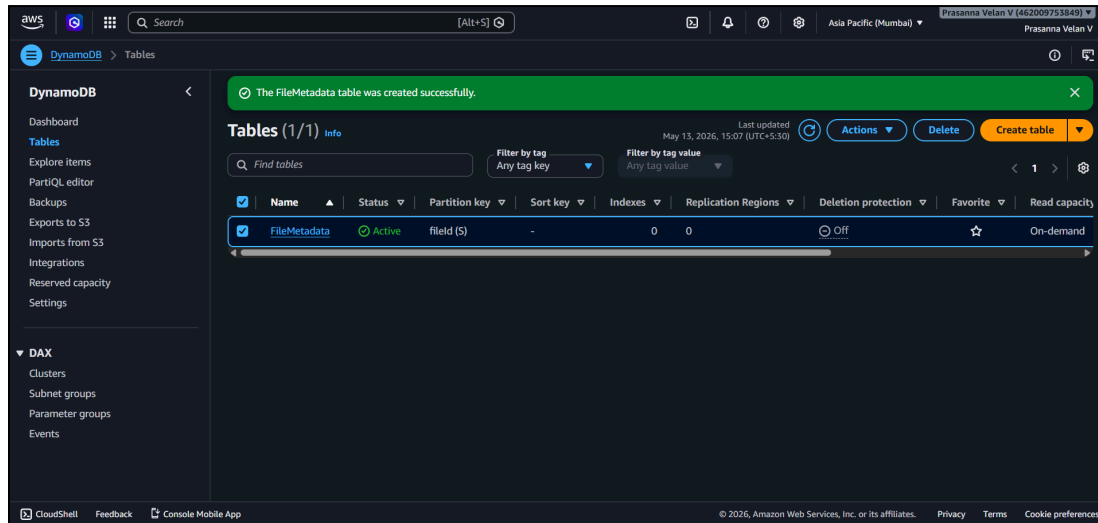
Step 1: Create Amazon S3 Bucket

- Open AWS Console → Amazon S3.
- Click Create Bucket.
- Enter bucket name - project-serverless-upload-bucket
- Select Region.
- Keep Block Public Access enabled.
- Create uploads/ folder inside the bucket.



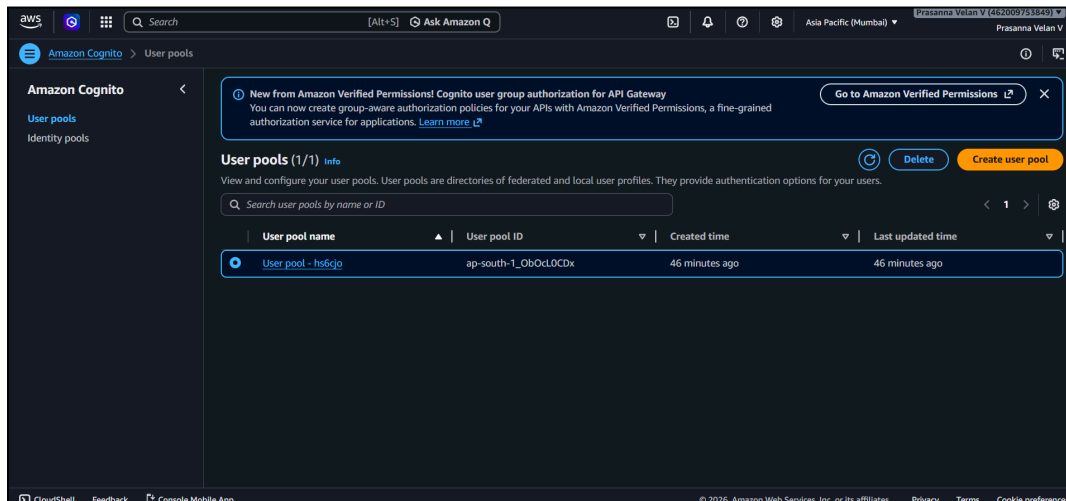
Step 2: Create DynamoDB Table

- Open AWS Console → DynamoDB.
- Click Create Table.
- Table Name: FileMetadata.
- Partition Key: fileId (String).
- Keep default settings.
- Create the table.



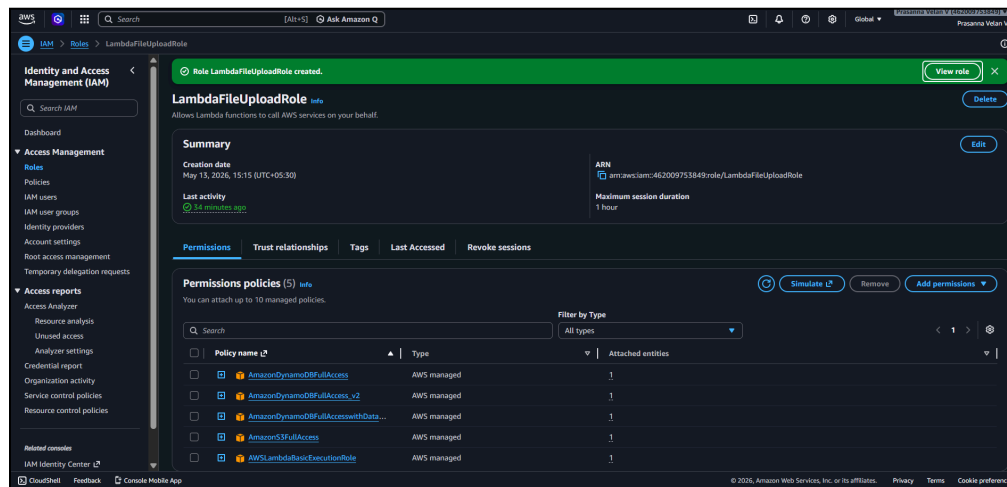
Step 3: Create Cognito User Pool

- Open AWS Console → Amazon Cognito.
 - Create User Pool.
- Select Single Page Application (SPA).
 - Enable Email sign-in.
 - Enable self-registration.
- Configure Hosted UI login page.
- Generate User Pool ID and App Client ID.



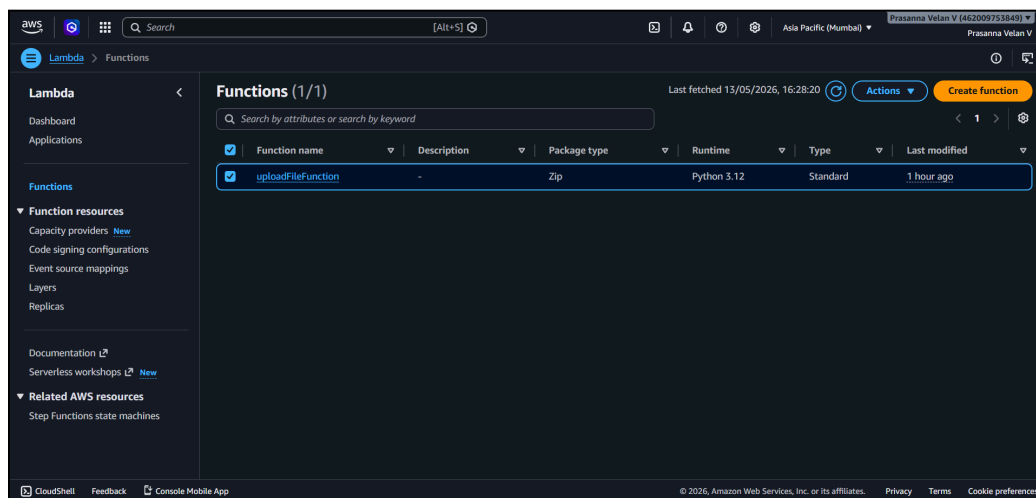
Step 4: Create IAM Role for Lambda

- Open AWS Console → IAM → Roles.
- Create Role for Lambda service.
- Attach policies:
 - AWSLambdaBasicExecutionRole
 - AmazonS3FullAccess
 - AmazonDynamoDBFullAccess
 - Role Name: LambdaFileUploadRole.



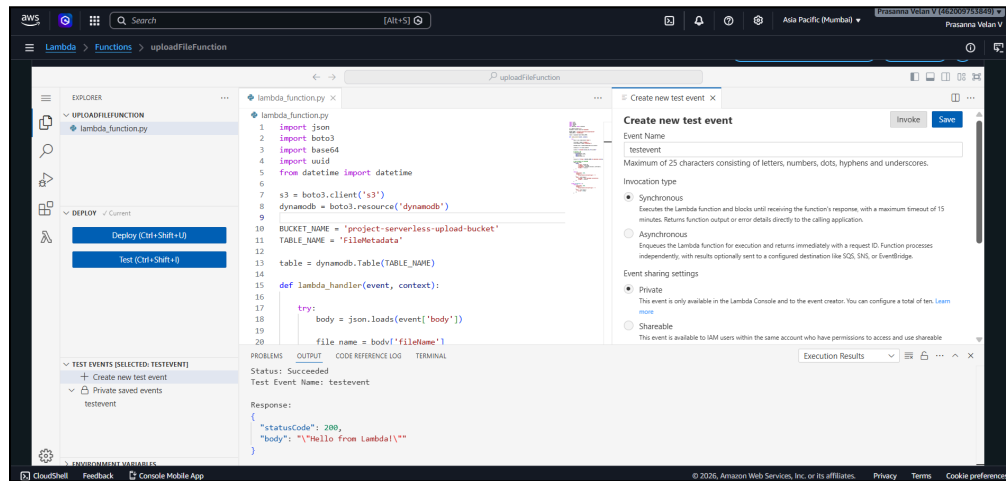
Step 5: Create Lambda Function

- Open AWS Console → Lambda.
- Click Create Function.
- Function Name: uploadFileFunction.
- Runtime: Python 3.12.
- Attach Existing IAM Role: LambdaFileUploadRole.



Step 6: Add Lambda Function Code and Test Lambda Function

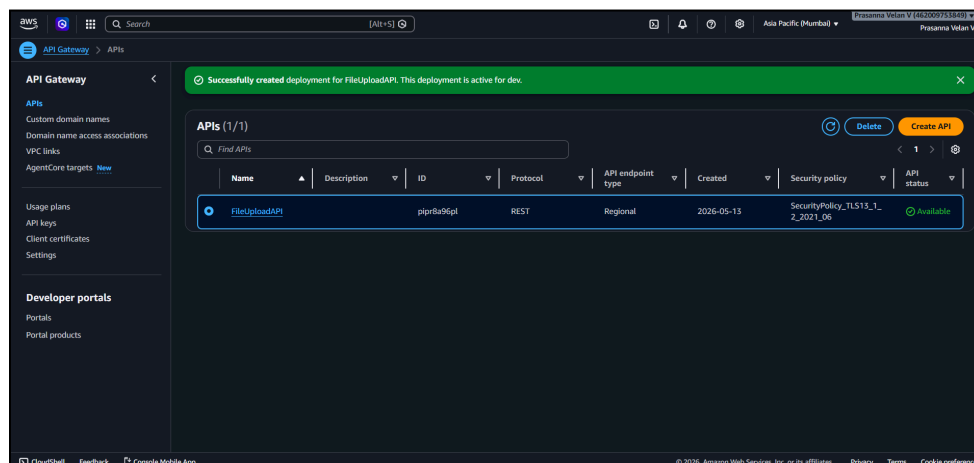
- Add Python code for file upload processing.
- Configure S3 bucket name and DynamoDB table name.
- Save and Deploy Lambda function.



- Create test event in Lambda console.
- Execute test event.
- Verify successful response.
- Confirm files uploaded in S3.
- Confirm metadata stored in DynamoDB.

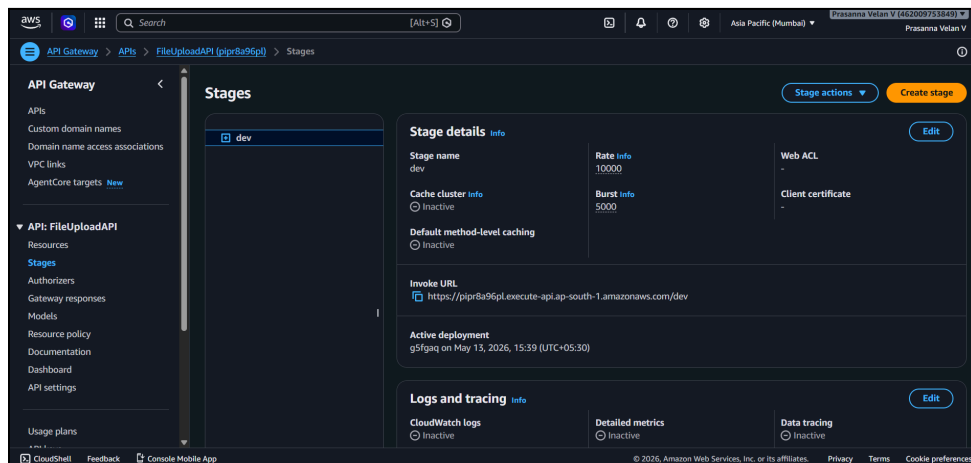
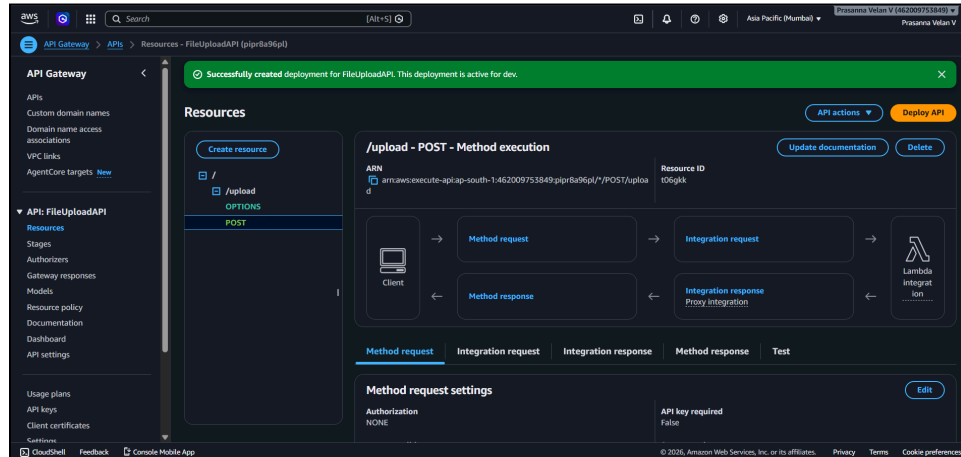
Step 7: Create API Gateway REST API

- Open AWS Console → API Gateway.
- Create REST API.
- API Name: FileUploadAPI.
- Create Resource: /upload and Create POST method.
- Integrate POST method with Lambda function uploadFileFunction.



Step 8: Enable CORS and Deploy API Gateway

- Open API Gateway → Resources.
- Select /upload resource.
- Enable CORS for POST method.
- Allow browser access using Access-Control-Allow-Origin



- Click Deploy API.
- Create Stage: dev.
- Deploy API.
- Copy Invoke URL for frontend integration.

Step 9: Create Frontend Application

- Create index.html file.
- Build frontend using HTML, CSS, and JavaScript.
- Add file upload functionality.
- Integrate API Gateway endpoint using fetch().
- Add responsive and modern UI design.

Serverless Web Application with AWS Lambda and API Gateway

The screenshot shows a VS Code editor with the following content:

```

137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172

<body>

  <div class="container">

    <div class="logo">
      <h1>AWS File Upload</h1>
      <p>Serverless Web Application using AWS</p>
    </div>

    <div class="upload-box">
      <h3>Select file</h3>

      <input type="file" id="fileInput" onchange="showFileName()" />

      <div class="file-name" id="fileName">
        | no file selected
      </div>
    </div>

    <button id="uploadBtn" onclick="uploadFile()">
      | Upload File
    </button>

    <div class="loader" id="loader"></div>

    <div class="status" id="status"></div>

    <div class="footer">
      | Powered by AWS Lambda • API Gateway • S3 • DynamoDB
    </div>
  </div>
</body>
</script>

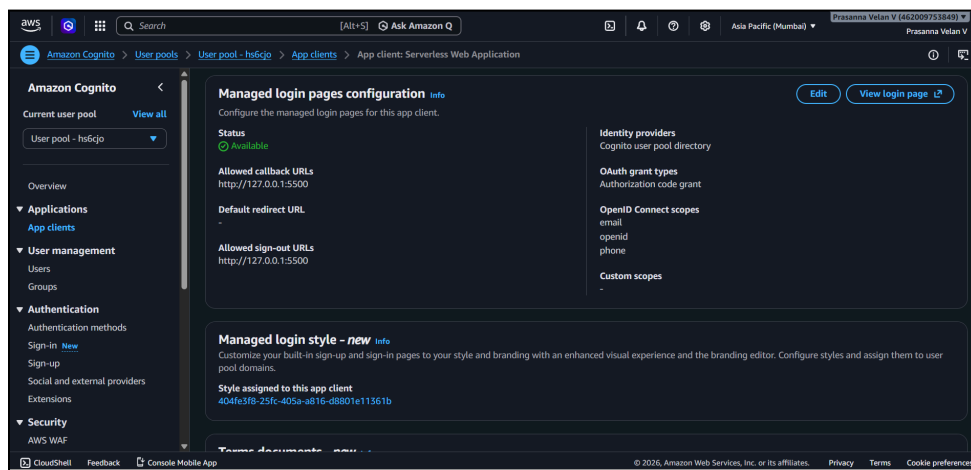
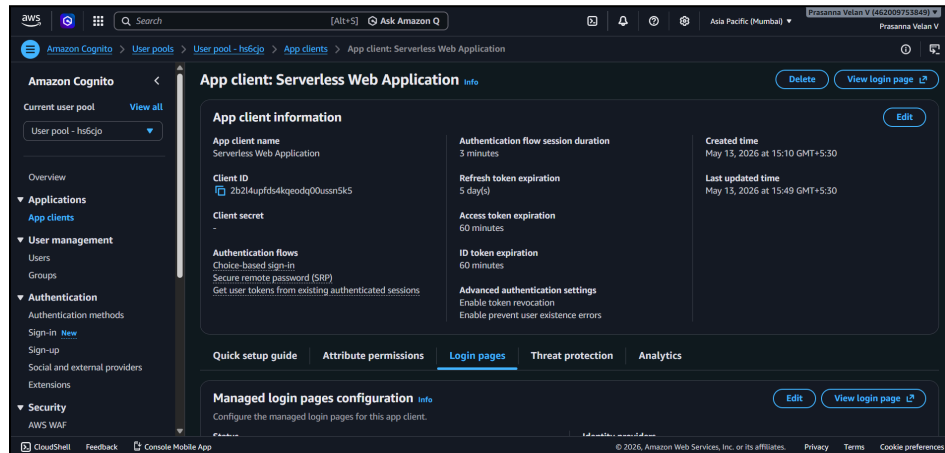
```

The footer text is: "Powered by AWS Lambda • API Gateway • S3 • DynamoDB".

Step 10: Add Cognito Authentication

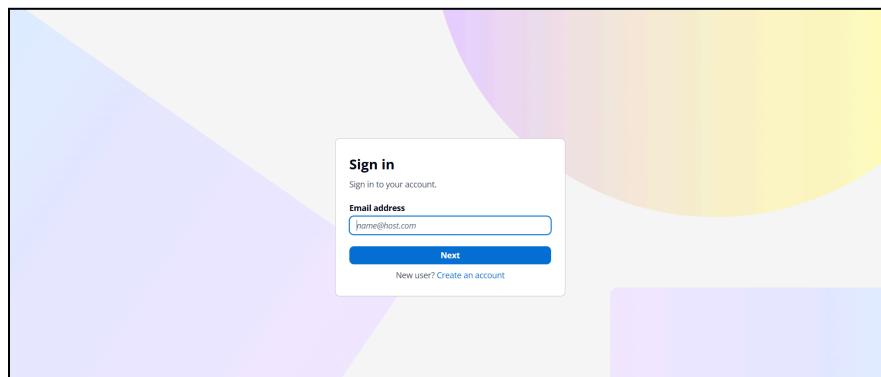
- Configure Cognito Hosted UI.
- Add Callback URL and Sign-out URL.
- Enable Cognito authentication provider.
- Add Login with Cognito button in frontend.
- Authenticate users before accessing upload functionality.

Serverless Web Application with AWS Lambda and API Gateway



Step 11: Test Complete Application Flow

- User signs up using Cognito Hosted UI.
- User logs in successfully.
- User selects file from frontend.
- File uploads to Amazon S3.
- Metadata stores in DynamoDB.
- API response returned successfully.



Serverless Web Application with AWS Lambda and API Gateway

Sign up

Create a new account.

Email address

prasanavelan2003@gmail.com

Password

jeyavel@123

Password must be at least 8 characters

Use a number

Use a lowercase letter

Use an uppercase letter

Use a special character

Confirm password

jeyavel@123

Show password

Sign up

Have an account already? Sign in

AWS File Upload

Serverless Web Application using AWS

Select File

Choose file

No file chosen

No file selected

Upload File

Powered by AWS Lambda • API Gateway • S3 • DynamoDB

AWS File Upload

Serverless Web Application using AWS

Select File

Choose file

Prasanna_V_Resume.pdf

Prasanna_Velari_V_Resume.pdf

Upload File

Powered by AWS Lambda • API Gateway • S3 • DynamoDB

AWS File Upload

Serverless Web Application using AWS

Select File

Choose file

Prasanna_V_Resume.pdf

Prasanna_Velari_V_Resume.pdf

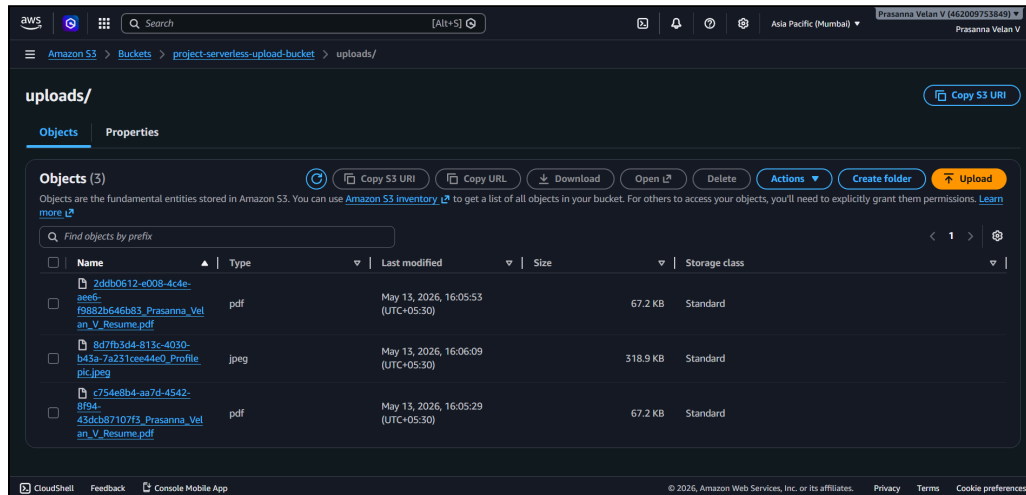
Upload File

File uploaded successfully

View Uploaded File

Powered by AWS Lambda • API Gateway • S3 • DynamoDB

Serverless Web Application with AWS Lambda and API Gateway



Troubleshooting:

Issue 1: CORS Error in Browser

The browser blocks API requests because CORS headers are not configured correctly.

Solution:

Enable CORS in API Gateway.

- Open API Gateway → Resources.
- Select POST method.
- Enable CORS.
- Allow origin using “*”.
- Redeploy API.

Issue 2: Access Denied Error for S3 Upload

Lambda function does not have permission to upload files to S3.

Solution:

Attach correct IAM permissions.

- Open IAM → Lambda Role.
- Attach AmazonS3FullAccess policy.
- Save changes.

Issue 3: Lambda Timeout Error

Lambda execution timeout is too short for large file uploads.

Solution:

Increase Lambda timeout.

- Open Lambda → Configuration.
- Select General Configuration.
- Edit Timeout.
- Set Timeout to 30 seconds.

Issue 4: API Gateway Internal Server Error

Lambda function returns errors because of incorrect bucket names or missing permissions.

Solution:

- Verify S3 bucket name in Lambda code.
- Verify DynamoDB table name.
- Check CloudWatch Logs for detailed errors.

Conclusion:

The Serverless File Upload Web Application provides a secure, scalable, and fully serverless solution for authenticated file uploads using AWS managed services. The project successfully integrates Amazon Cognito for authentication, API Gateway for REST API management, AWS Lambda for backend processing, Amazon S3 for file storage, and DynamoDB for metadata management.

The application eliminates the need for server management, reduces operational costs, and provides high scalability and availability through AWS serverless architecture.

This project demonstrates practical implementation of modern cloud-native application development and is suitable for organizational deployment, academic projects, portfolio showcases, and real-world serverless use cases.